DESARROLLO WEB EN ENTORNO SERVIDOR

UT1. PROYECTO DE APLICACIÓN DE GENERACIÓN DE INFRAESTRUCTURAS CON IA

PARTE A: ENTORNO DE DESARROLLO Y CÓDIGO FUENTE INICIAI

Fase 1 · Preparación reproducible de Ubuntu, Laravel y herramientas asociadas

Nombre del proyecto	***Cloud 9 Systems***
Integrantes	Yoel José Bautista Díaz, Ángel León Santana, Juan Francisco Millet Acedo, Damián Pérez Alemán
Módulo	Desarrollo Web en Entorno Servidor
Curso	2026/2027
Centro	CIFP VILLA DE AGÜIMES
Fecha de entrega	14 / 10 / 2026
Profesor	YERAY ANDRÉS PÉREZ VALIENTE

Engineering your cloud.

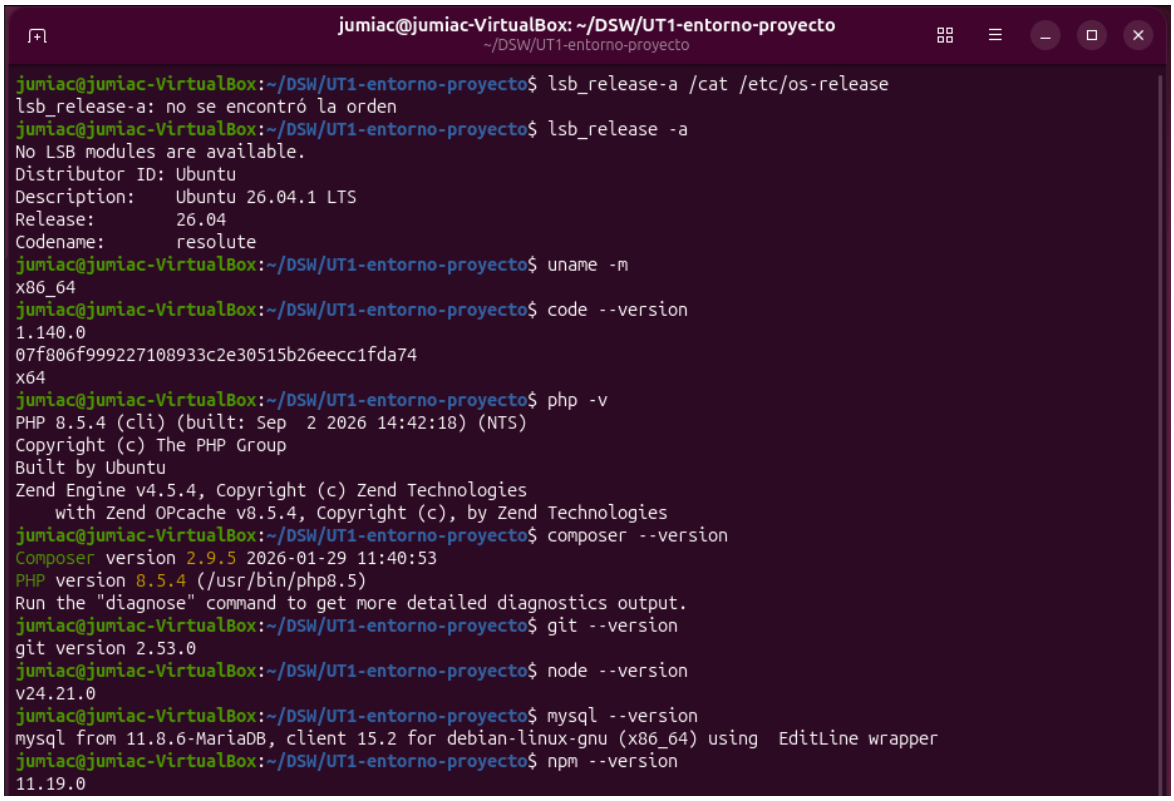
ÍNDICE

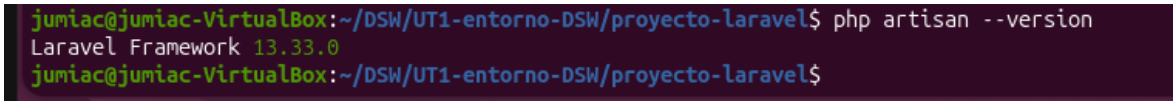
ÍNDICE 2
1. Resumen del entorno preparado 4
1.1 Datos de la máquina 5
2. Fases seguidas para preparar el entorno 5
3. Sistema operativo Ubuntu 6
3.1 Identificación del sistema 6
3.2 Actualización inicial 6
3.3 Incidencias y decisiones 7
4. Visual Studio Code 7
4.1 Función dentro del entorno 7
4.2 Extensiones instaladas y justificación (en total son muchas las importantes) 7
4.3 Terminal integrada y Git 8
5. PHP y extensiones 8
5.1 Conceptos mínimos 8
5.2 Verificación de versión 9
5.3 Extensiones PHP instaladas 9
6. Composer y dependencias PHP 10
6.1 Conceptos mínimos 10
Extensión 10
Necesidad / función 10
Verificación 10
php-mbstring 10
Permite gestionar cadenas de texto multibyte, algo fundamental para que Laravel maneje caracteres especiales e idiomas correctamente 10
php -m | grep mbstring (Muestra el módulo activo) 10
php-xml 10
Se encarga de procesar y estructurar documentos XML, necesarios para librerías internas de Laravel y el gestor de dependencias Composer 10
php -m | grep xml (Confirma la instalación) 10
php-mysql 10
Proporciona los controladores de datos para que la aplicación PHP pueda conectarse y hacer consultas a la base de datos MariaDB/MySQL 10
php -m | grep pdo_mysql (Verifica el driver PDO) 10
php-curl 10
Permite al servidor realizar peticiones HTTP externas, algo obligatorio para conectar de forma fluida el backend con la API de la Inteligencia Artificial 10
php -m | grep curl (Muestra la extensión cargada) 10
6.2 Verificación 11
7. Git y GitHub 11

7.1 Conceptos Git....11
7.2 Verificación de Git....12
7.3 Repositorio compartido....12
7.4 Ramas utilizadas....12
7.5 Participación de los integrantes....13
8. Node.js, npm y Vite....14
8.1 Verificación....14
9. Servidor de base de datos....15
9.1 Configuración utilizada....15
9.2 Función del fichero .env....15
9.3 Prueba de funcionamiento....16
10. Creación inicial del proyecto Laravel....17
10.1 Nombre y creación....17
10.2 Verificación de Laravel....17
10.3 Puesta en marcha....17
10.4 Primera página de comprobación....18
10.5 Explicación breve del ciclo MVC demostrado....18
11. Seguridad inicial del repositorio....19
11.1 Fichero .env....19
11.2 .gitignore....19
11.3 .env.example....19
12. Estructura de entrega dentro del repositorio....20
12.1 Estructura real del equipo....20
12.2 Estado actual del código fuente....21
13. README obligatorio....21
14. Procedimiento de reproducción del entorno....22
14.1 Prueba final de reproducibilidad....23
15. Evidencias técnicas....24
Evidencia 1. Ubuntu funcionando y versión del sistema....24
Evidencia 2. PHP y extensiones necesarias....24
Evidencia 3. Composer....25
Evidencia 4. Git....25
Evidencia 5. Node.js y npm....26
Evidencia 6. Servidor de base de datos....26
Evidencia 7. Laravel....28
Evidencia 8. Proyecto ejecutándose....28
Evidencia 9. Página inicial de comprobación....29
Evidencia 10. Repositorio GitHub....30
Evidencia 11. Ramas principales y feature....32
Evidencia 12. Commits identificables de los integrantes....33
16. Incidencias encontradas y resolución....34
17. Conclusiones técnicas....34
18. Checklist antes de entregar....35

1. Resumen del entorno preparado

Componente	Versión instalada	Estado	Comando de verificación
Ubuntu	Ubuntu 26.04.1 LTS	☑ OK	lsb_release -a
Arquitectura	x86_64	☑ OK	uname -m
Visual Studio Code	1.140.0	☑ OK	code --version
PHP	8.5.4	☑ OK	php -v
Composer	2.9.5	☑ OK	composer --version
Git	2.53.0	☑ OK	git --version
Node.js	v24.12.0	☑ OK	node --version
npm	11.19.0	☑ OK	npm --version
Servidor BD	11.8.6-MariaDB	☑ OK	mysql --version
Laravel	Laravel Framework 13.33.0	☑ OK	php artisan --version





1.1 Datos de la máquina

Dato	Valor
Hostname	jumiac-VirtualBox
Usuario de desarrollo	jumiac
Tipo de máquina	☑ VM
CPU / RAM asignada si es VM	2 núcleos CPU / 4 GB RAM *(3.3 GiB reales)*
Dirección IP si procede	10.0.2.15 (Red NAT)

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ whoami && hostname
jumiac
jumiac-VirtualBox
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ hostname -I
10.0.2.15 fd17:625c:f037:2:a00:27ff:fed9:e34b
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ free --human
               total        usado       libre  compartido   búf/caché  disponible
Mem:           3,3Gi       1,1Gi       762Mi        22Mi       1,7Gi       2,2Gi
Inter:         3,8Gi       481Mi       3,3Gi
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ nproc
2
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ hostname -I
10.0.2.15 fd17:625c:f037:2:a00:27ff:fed9:e34b
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$
```

2. Fases seguidas para preparar el entorno

Fase	Objetivo	Acciones principales	Resultado
1	Preparar Ubuntu	Actualizar repositorios del sistema y configurar la máquina virtual.	Operativo el 25/09/2026
2	Instalar editor y herramientas base	Descargar e instalar Visual Studio Code mediante comandos snap o paquetes deb.	Operativo el 26/09/2026
3	Instalar PHP y extensiones	Añadir repositorio de PHP, instalar la versión base y las extensiones necesarias para Laravel (mbstring, xml, etc.).	Operativo el 28/09/2026
4	Instalar Composer	Descargar el instalador oficial, verificar el hash y mover el ejecutable a la ruta global del sistema.	Operativo el 29/09/2026
5	Instalar Git y configurar GitHub	Instalar el paquete de Git, configurar el nombre de usuario/email y añadir las claves SSH.	Operativo el 01/10/2026
6	Instalar Node.js/npm	Descargar e instalar Node.js y su gestor de paquetes npm para la compilación de recursos del frontend.	Operativo el 02/10/2026
7	Instalar y configurar BD	Instalar el servidor MariaDB/MySQL, asegurar la instalación y crear el usuario de desarrollo.	Operativo el 03/10/2026
8	Crear y comprobar Laravel	Crear el nuevo proyecto con Composer y arrancar el servidor con php artisan server para ver la pantalla de bienvenida.	Operativo el 04/10/2026

3. Sistema operativo Ubuntu

3.1 Identificación del sistema

Aspecto	Información
Versión instalada	Ubuntu 26.04.1 LTS
Arquitectura	x86_64
Hostname	jumiac-VirtualBox
Usuario de desarrollo	jumiac
Método de instalación / origen de la VM	Instalación limpia mediante ISO oficial en VirtualBox / Imagen base de clase

3.2 Actualización inicial

Comandos utilizados:

sudo apt update

sudo apt upgrade -y

Explicación de lo realizado y resultado:

Se ejecutaron los comandos de actualización de los repositorios del sistema para sincronizar las últimas versiones de los paquetes disponibles. Posteriormente, se aplicó la actualización de todo el software del sistema operativo instalado en la máquina virtual. El proceso finalizó con éxito de manera limpia, dejando todas las dependencias base de Ubuntu totalmente al día y preparadas para las instalaciones del entorno de desarrollo.

3.3 Incidencias y decisiones

Incidencia / decisión	Causa	Solución aplicada	Cómo se verificó
Error al ejecutar php artisan en la terminal	Intentar lanzar el comando desde la raíz del usuario (~), donde el archivo artisan no existe.	Cambiar de directorio usando cd para situarse dentro de la carpeta del proyecto Laravel.	Al ejecutar php artisan --version devolvió el número correcto de versión del framework.

4. Visual Studio Code

4.1 Función dentro del entorno

Visual Studio Code actúa como el entorno de desarrollo integrado (IDE) principal del proyecto. Aporta una interfaz ligera y altamente personalizable que centraliza la edición de código, la gestión del sistema de archivos de Laravel y el acceso directo a la terminal de Ubuntu. Gracias a su ecosistema de extensiones, transforma un editor de texto plano en una herramienta potente capaz de entender la sintaxis de PHP, autocompletar funciones del framework, formatear plantillas Blade automáticamente y gestionar el control de versiones con Git de manera visual, lo que optimiza drásticamente la velocidad de desarrollo y reduce errores de sintaxis.

4.2 Extensiones instaladas y justificación (en total son muchas las importantes)

Extensión	Función	Por qué se ha instalado	¿Obligatoria o opcional?
PHP IntelliSense	Proporciona autocompletado avanzado, navegación de código y sugerencias inteligentes para PHP.	Para agilizar la escritura de código backend y evitar errores en nombres de clases o métodos.	Obligatoria
Laravel Artisan	Permite ejecutar comandos de Laravel Artisan directamente desde la paleta de comandos de VS Code.	Para agilizar la creación de modelos, controladores y migraciones sin necesidad de cambiar continuamente a la terminal.	Opcional (Recomendada)
Laravel Blade Snippets / Formatter	Añade resaltado de sintaxis, autocompletado y formateo automático para las vistas .blade.php.	Para que el código del frontend sea legible, mantenga una sangría limpia y se autocompleten las directivas de Laravel.	Obligatoria

4.3 Terminal integrada y Git

La terminal integrada de Visual Studio Code se utilizará como la herramienta principal para interactuar con el backend de Laravel y el sistema operativo sin necesidad de salir del editor. A través de ella, se ejecutarán de forma constante los comandos de Artisan para levantar el servidor local (php artisan serve), aplicar las migraciones de la base de datos y generar los componentes lógicos del sistema (modelos, servicios y controladores).

Respecto al control de versiones, se combinará el uso de la línea de comandos con la interfaz visual de Git integrada en VS Code:

- **Terminal integrada:** Se empleará para tareas rápidas y precisas mediante comandos nativos como git pull, git add . y git commit -m "..." siguiendo la convención de mensajes establecida.
- **Interfaz gráfica de VS Code:** Se utilizará de apoyo visual para revisar las diferencias de código línea por línea antes de confirmar un cambio (*diff*), resolver conflictos de fusionado de ramas de manera intuitiva y gestionar de forma segura la sincronización con el repositorio remoto de GitHub.

5. PHP y extensiones

5.1 Conceptos mínimos

Concepto	Explicación del equipo
¿Qué es PHP?	Es un lenguaje de programación de código abierto que se utiliza principalmente para el desarrollo web en el lado del servidor, permitiendo crear páginas y aplicaciones dinámicas.
¿Dónde se ejecuta?	Se ejecuta exclusivamente en el servidor web. El servidor procesa el código PHP y le envía al navegador del usuario un resultado final en formato HTML limpio.
Diferencia respecto a JavaScript en navegador	PHP corre en el servidor (backend) antes de que la página se cargue, controlando bases de datos y seguridad. JavaScript corre directamente en el navegador del usuario (frontend) para manejar la interactividad visual de la página una vez cargada.
Relación entre PHP y Laravel	PHP es el lenguaje de programación base, mientras que Laravel es un framework (un conjunto de herramientas, librerías y reglas) construido sobre PHP que facilita y acelera el desarrollo de aplicaciones web organizadas.
¿Qué es una extensión PHP?	Es un módulo o librería adicional que se añade a la instalación principal de PHP para darle nuevas funciones, como conectarse a ciertos tipos de bases de datos, cifrar datos o procesar imágenes.

5.2 Verificación de versión

```
php -v
```

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ php -v
PHP 8.5.4 (cli) (built: Sep  2 2026 14:42:18) (NTS)
Copyright (c) The PHP Group
Built by Ubuntu
Zend Engine v4.5.4, Copyright (c) Zend Technologies
    with Zend OPcache v8.5.4, Copyright (c), by Zend Technologies
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$
```

5.3 Extensiones PHP instaladas

Extensión	Necesidad / función	Verificación
php-mbstring	Permite gestionar cadenas de texto multibyte, algo fundamental para que Laravel maneje caracteres especiales e idiomas correctamente.	php -m | grep mbstring (Muestra el módulo activo).
php-xml	Se encarga de procesar y estructurar documentos XML, necesarios para librerías internas de Laravel y el gestor de dependencias Composer.	php -m | grep xml (Confirma la instalación).
php-mysql	Proporciona los controladores de datos para que la aplicación PHP pueda conectarse y hacer consultas a la base de datos MariaDB/MySQL.	php -m | grep pdo_mysql (Verifica el driver PDO).
php-curl	Permite al servidor realizar peticiones HTTP externas, algo obligatorio para conectar de forma fluida el backend con la API de la Inteligencia Artificial.	php -m | grep curl (Muestra la extensión cargada).

6. Composer y dependencias PHP

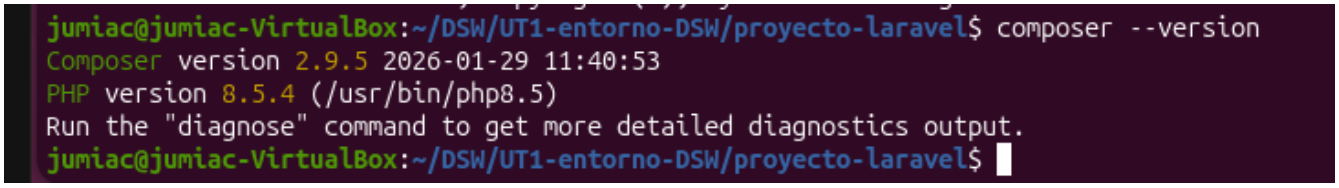
6.1 Conceptos mínimos

Extensión	Necesidad / función	Verificación
php-mbstring	Permite gestionar cadenas de texto multibyte, algo fundamental para que Laravel maneje caracteres especiales e idiomas correctamente.	php -m | grep mbstring (Muestra el módulo activo).

php-xml	Se encarga de procesar y estructurar documentos XML, necesarios para librerías internas de Laravel y el gestor de dependencias Composer.	php -m | grep xml (Confirma la instalación).
php-mysql	Proporciona los controladores de datos para que la aplicación PHP pueda conectarse y hacer consultas a la base de datos MariaDB/MySQL.	php -m | grep pdo_mysql (Verifica el driver PDO).
php-curl	Permite al servidor realizar peticiones HTTP externas, algo obligatorio para conectar de forma fluida el backend con la API de la Inteligencia Artificial.	php -m | grep curl (Muestra la extensión cargada).

6.2 Verificación

```
composer --version
```



7. Git y GitHub

7.1 Conceptos Git

Concepto	Definición breve del equipo
Repositorio	Carpeta o espacio virtual estructurado donde Git guarda todos los archivos del proyecto junto con el historial completo de sus modificaciones.
Commit	Guardado o instantánea que registra los cambios realizados en el código en un momento concreto junto con un mensaje descriptivo.
Rama	Línea de desarrollo independiente que permite trabajar en una nueva funcionalidad o corrección sin alterar el código principal del proyecto.

Concepto	Definición breve del equipo
Merge	Operación que une y fusiona los cambios realizados en una rama secundaria dentro de otra rama de destino (como la rama principal).
Remote	Versión del repositorio que está alojada de forma externa en un servidor de internet o nube compartida, como en la plataforma GitHub.
Push	Comando que se utiliza para subir y enviar los commits guardados en tu máquina local hacia el repositorio remoto en la nube.
Pull	Comando que sirve para descargar y fusionar los cambios más recientes que otros compañeros hayan subido al repositorio remoto.
Clone	Proceso que descarga una copia exacta por primera vez de un repositorio remoto de internet hacia tu carpeta local de desarrollo.
Conflicto	Problema que ocurre cuando dos personas modifican la misma línea de un archivo y Git no sabe de forma automática qué versión debe prevalecer.
Pull Request	Petición formal para revisar, debatir y aprobar la fusión de los cambios de una rama antes de unirlos al código principal del proyecto.

7.2 Verificación de Git

```
git --version
```

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ git --version
git version 2.53.0
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$
```

7.3 Repositorio compartido

URL del repositorio	https://github.com/JuanMiAc/DSW_CLOUD_AJDJ
Visibilidad	Public
Acceso del profesor	Si
Miembros con acceso	Juan, Damián, Yoel, Angel

7.4 Ramas utilizadas

Rama	Finalidad	Responsables / contribuciones	Estado
main	Código de producción definitivo y completamente estable del proyecto.	Todo el equipo / Integración final	Activa (Estable)
rama_develop	Rama central de desarrollo donde se van uniendo todas las nuevas funcionalidades.	Todo el equipo / Base del backend	Activa (En desarrollo)
rama_test	Entorno de pruebas para verificar que el código funciona bien antes de pasarlo a main.	Todo el equipo / Control de calidad	Activa (En pruebas)
rama_juan_bd	Desarrollo específico del modelado, esquemas y bases de datos iniciales.	Juan Francisco Millet Acedo	Activa
feature/entorno	Configuración inicial de la máquina virtual, dependencias y arranque del proyecto base.	Damián Pérez Alemán	Fusionada y cerrada

7.5 Participación de los integrantes

Integrante	Ejemplos de commits / ramas	Aportación identificable
Juan Francisco	rama_juan_bd docs(db): diseñar diagrama conceptual entidad-relación feat(db): estructurar tablas iniciales del catálogo	Diseño conceptual de la base de datos, modelado de las relaciones de los 12 módulos funcionales y resolución lógica de la estructura de datos.
Damián	rama_develop docs(architecture): definir clases del service pattern refactor(models): unificar relaciones de proyectos	Planificación de la arquitectura limpia en el backend, diseño del flujo de las Clases de Servicio y delimitación de las reglas de negocio estrictas.

Integrante	Ejemplos de commits / ramas	Aportación identificable
Yoel	feature/frontend feat(ui): diseñar vistas y bocetos de la interfaz style(css): maquetar estructura del catálogo cloud	Diseño de la interfaz de usuario, desarrollo del frontend conceptual y preparación de los bocetos de las vistas del sistema.
Ángel	feature/backend feat(api): configurar controladores base para auth fix(core): resolver controladores de lógica de negócio	Desarrollo del código del servidor, configuración inicial de los controladores del backend y c

8. Node.js, npm y Vite

Concepto	Explicación del equipo
¿Por qué puede necesitar Node.js un proyecto Laravel?	Porque aunque Laravel funciona con PHP en el servidor (backend), necesita Node.js en el entorno de desarrollo para procesar, compilar y optimizar todos los archivos y librerías que forman la interfaz visual (frontend).
npm	Es el gestor de paquetes que viene con Node.js. Se encarga de descargar, instalar y actualizar automáticamente las librerías de JavaScript o CSS que vayamos a usar en la aplicación web.
Dependencias frontend	Son todos los paquetes de código externos que se necesitan para montar la parte visual y la interactividad del navegador, como frameworks de diseño (Tailwind CSS o Bootstrap) o librerías de componentes interactivos.
package.json	Es el archivo de configuración del frontend del proyecto. Contiene la lista con los nombres y las versiones exactas de todas las dependencias de JavaScript y CSS que utiliza la aplicación.
Vite	Es una herramienta moderna y súper rápida que empaqueta y compila los archivos del frontend. Recarga la vista automáticamente en el navegador en cuanto guardamos un cambio, facilitando el desarrollo rápido.

8.1 Verificación

```
node --version
npm --version
```

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ node --version
v24.21.0
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ npm --version
11.19.0
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$
```

Herramienta	Versión obtenida
Node.js	v24.12.0
npm	11.19.0

9. Servidor de base de datos

9.1 Configuración utilizada

Dato	Valor / explicación
Servidor de BD	MariaDB
Versión	11.8.6-MariaDB
Puerto	3306 *(confirmado por variable de sistema)*
Base de datos del proyecto	En preparación / Se creará mediante forge o comando SQL previo a las migraciones
Usuario de desarrollo	root *(confirmado por terminal en localhost)*
Conexión desde Laravel	Se realiza a través del archivo de entorno .env en la raíz del proyecto, mapeando los datos mediante el driver mysql y gestionando las tablas a través de las migraciones de Laravel.

9.2 Función del fichero .env

El archivo .env (Variables de Entorno) almacena todas las configuraciones críticas, credenciales de acceso y datos sensibles necesarios para que el proyecto funcione de manera local. Esto incluye los datos de conexión a la base de datos (puerto, usuario, contraseña), las claves de cifrado de la aplicación, configuraciones del servidor de correo y los tokens de acceso para servicios externos como la API de Inteligencia Artificial.

Este archivo cambia entre entornos porque los datos de conexión no son los mismos: en desarrollo local se usa un servidor de pruebas (como MariaDB en VirtualBox con usuario root), mientras que en el servidor de producción real los accesos son totalmente distintos y mucho más restrictivos.

Bajo ninguna circunstancia debe publicarse ni subirse al repositorio Git. Al contener contraseñas y claves privadas reales, subirlo a internet dejaría la aplicación totalmente expuesta a hackeos, robos de información o un uso fraudulento de servicios de pago como la IA. Por ello, se incluye siempre en el archivo .gitignore y se proporciona únicamente la plantilla .env.example vacía para el resto del equipo

9.3 Prueba de funcionamiento

Comprobación del estado del servicio de la base de datos en la terminal de Ubuntu mediante el comando:

sudo systemctl status mariadb

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ sudo systemctl status mariadb
● mariadb.service - MariaDB 11.8.6 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-10-05 15:42:28 CEST; 6 days ago
 Invocation: 98882351823a4d0f95fbb1c89d3b15d2
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
   Main PID: 1630 (mariadbd)
     Status: "Taking your SQL requests now..."
      Tasks: 11 (limit: 12102)
     Memory: 24.5M (peak: 138.9M, swap: 91.4M, swap peak: 91.4M)
        CPU: 4.671s
     CGroup: /system.slice/mariadb.service
             └─1630 /usr/sbin/mariadbd
```

Y verificación de la conectividad interna y las variables del servidor mediante el comando:

sudo mysql -u root -e "SHOW VARIABLES LIKE 'port'; SELECT USER(); SHOW DATABASES;"

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ sudo mysql -u root -e "SHOW VARIABLES LIKE 'port'
; SELECT USER(); SHOW DATABASES;"
[sudo: authenticate] Contraseña:
+---------------+-------+
| Variable_name | Value |
+---------------+-------+
| port          | 3306  |
+---------------+-------+
+----------------+
| USER()         |
+----------------+
| root@localhost |
+----------------+
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$
```

Resultado:

El sistema gestor de bases de datos responde correctamente y se encuentra activo (*active/running*). La terminal confirma que el servidor escucha en el puerto estándar **3306**, permite el acceso local al usuario **root** de forma segura y devuelve el listado inicial de tablas del sistema sin errores de comunicación.

10. Creación inicial del proyecto Laravel

10.1 Nombre y creación

Dato	Información
Nombre del proyecto Laravel	proyecto-laravel
Directorio	~/DSW/UT1-entorno-DSW/proyecto-laravel *(o la ruta exacta de tu máquina virtual)*
Comando de creación	composer create-project laravel/laravel proyecto-laravel

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ pwd
/home/jumiac/DSW/UT1-entorno-DSW/proyecto-laravel
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$
```

10.2 Verificación de Laravel

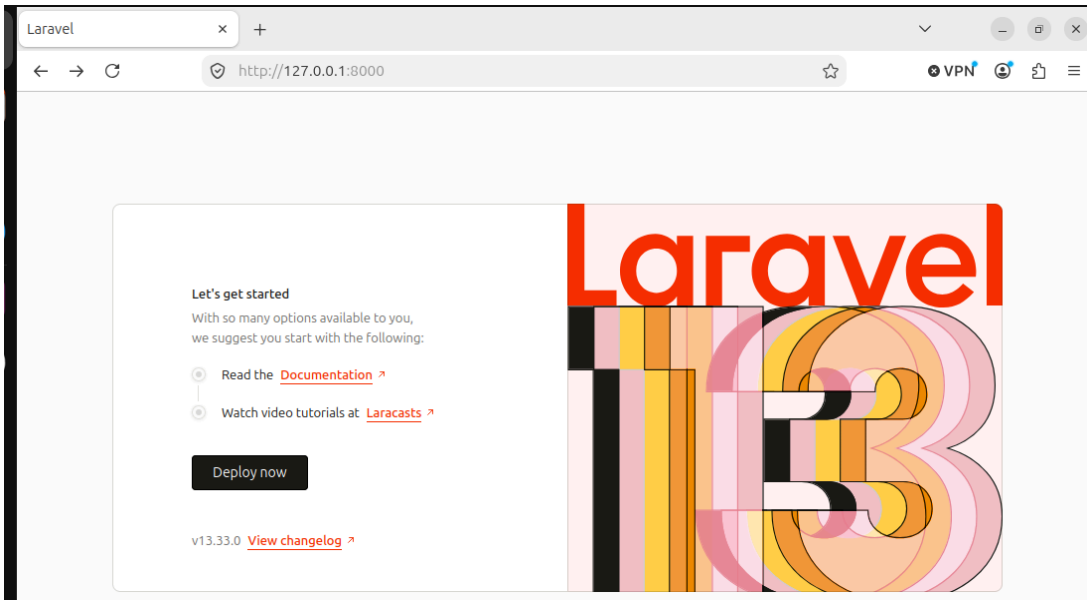
```
php artisan --version
```

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ php artisan --version
Laravel Framework 13.33.0
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$
```

10.3 Puesta en marcha

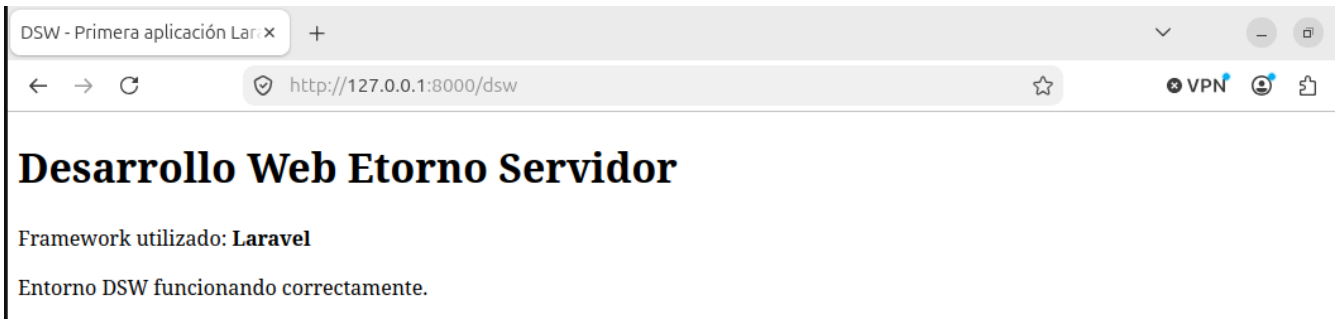
Comando utilizado para ejecutar el proyecto: *php artisan server*

URL utilizada para comprobarlo desde el navegador: *http://127.0.0.1:8000*



10.4 Primera página de comprobación

Elemento mostrado	Valor
Nombre de la aplicación	Desarrollo Web Entorno Servidor
Mensaje de entorno preparado	Entorno DSW funcionando correctamente.
Equipo	Juan Francisco, Damián, Yoel y Ángel
Versión Laravel	Laravel Framework 13.33.0
Versión PHP	PHP 8.5.4



10.5 Explicación breve del ciclo MVC demostrado

El flujo de la petición realizada en el navegador sigue el patrón **Modelo-Vista-Controlador (MVC)** de Laravel de la siguiente manera:

1. **Ruta (Route):** El navegador realiza una petición HTTP a la URL http://127.0.0. El archivo de rutas de Laravel (routes/web.php) recibe esta petición y busca la regla que coincida con el camino /dsw.
2. **Controlador / Código Servidor:** Al encontrar la ruta, Laravel ejecuta la lógica asociada (que puede estar directamente en la ruta o delegada en un Controlador). Este código se encarga de recopilar los datos necesarios que queremos mostrar en pantalla (como las versiones de PHP y Laravel o el mensaje de confirmación).
3. **Vista (View):** El controlador inyecta esos datos y llama a una vista estructurada en un archivo de plantilla (por ejemplo, dsw.blade.php). El motor Blade de Laravel procesa las variables de PHP en el servidor y genera un archivo HTML plano y limpio.
4. **Respuesta:** El servidor web envía ese HTML de vuelta al navegador del usuario, que lo interpreta y lo muestra de forma visual en la pantalla tal y como se ve en la captura.

11. Seguridad inicial del repositorio

11.1 Fichero .env

El archivo .env **nunca debe subirse a GitHub porque contiene las credenciales de acceso reales y las claves de cifrado del proyecto en desarrollo local**.

Publicarlo en un repositorio, incluso si es privado, implica **graves riesgos de seguridad**:

- Cualquier persona con acceso al repositorio podría robar las contraseñas de la base de datos o las API Keys de la Inteligencia Artificial, utilizándolas de forma fraudulenta y generando costes económicos elevados a nuestro nombre.
- Se expondría la APP_KEY de Laravel, que es la clave que usa el framework para cifrar las cookies, las sesiones de los usuarios y los tokens de seguridad. Si un atacante la consigue, podría hackear los accesos o suplantar identidades en la aplicación.

11.2 .gitignore

Comprobación	Resultado
.env está ignorado	☑ **Sí** ☐ No
No existen contraseñas en el repositorio	☑ **Comprobado**
No existen tokens/API Keys	☑ **Comprobado**
No existen claves privadas	☑ **Comprobado**

Comprobación	Resultado
No existen secretos de conexión	☑ **Comprobado**

11.3 .env.example

En el archivo .env.example se conservan **únicamente los nombres de las variables de configuración sin ningún valor real o secreto**. Se mantienen las claves estructurales del sistema como APP_ENV=local, las variables de conexión vacías (DB_CONNECTION=mysql, DB_HOST=127.0.0.1, DB_PORT=3306), y los nombres de los parámetros requeridos para la API de la IA.

Esto permite que, cuando un nuevo compañero clone el proyecto desde GitHub, solo tenga que hacer una copia de este archivo, renombrarlo a .env y **rellenar los campos con las contraseñas de su propia máquina virtual**. De esta forma, el proyecto se puede configurar y arrancar en segundos en cualquier entorno sin necesidad de publicar o compartir ningún secreto en internet.

12. Estructura de entrega dentro del repositorio

Mostrar la estructura real utilizada. Como mínimo deberá existir una carpeta docs y el proyecto Laravel inicial.

```
repositorio/
├── docs/
│   ├── entorno.md
│   ├── arquitectura.md
│   ├── requisitos.md
│   ├── casos-de-uso.md
│   ├── roles-equipo.md
│   └── planificacion.md
└── proyecto-laravel/
    ├── .env.example
    ├── .gitignore
    ├── README.md
    ├── composer.json
    └── package.json
```

12.1 Estructura real del equipo

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW$ tree -L 2
.
├── docs
│   ├── 01-arquitectura.md
│   ├── 02-tecnologias.md
│   ├── 03-entorno.md
│   ├── 04-git.md
│   └── entorno.txt
├── proyecto-laravel
│   ├── AGENTS.md
│   ├── app
│   ├── artisan
│   ├── bootstrap
│   ├── CLAUDE.md
│   ├── composer.json
│   ├── composer.lock
│   ├── config
│   ├── database
│   ├── package.json
│   ├── phpunit.xml
│   ├── public
│   ├── README.md
│   ├── resources
│   ├── routes
│   ├── storage
│   ├── tests
│   ├── vendor
│   └── vite.config.js
└── README.md

13 directories, 15 files
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW$
```

12.2 Estado actual del código fuente

A fecha de hoy, el código fuente de la aplicación se encuentra en su **fase inicial de despliegue de entorno**. Contiene únicamente la instalación limpia de la estructura nativa de directorios que genera Laravel, junto con las dependencias iniciales de backend de Composer y las herramientas de frontend gestionadas por npm y Vite.

En este punto del desarrollo, el código incluye exclusivamente la ruta /dsw configurada en el servidor y la lógica mínima para renderizar la **pequeña página de comprobación** que valida las versiones funcionales de PHP y del framework en el navegador. **No se ha implementado todavía ninguna de las funcionalidades operativas del proyecto anual**, tales como el asistente de IA, la base de datos de los 12 módulos funcionales o el procesamiento de presupuestos, cuya codificación e integración darán comienzo de manera oficial en la UT2.

13. README obligatorio

El README debe permitir que un desarrollador nuevo comprenda el estado actual y pueda poner el proyecto en funcionamiento.

Apartado README	¿Incluido?	Observaciones
Descripción	☑ **Sí**	Explica brevemente el propósito de la plataforma de simulación Cloud y el uso del asistente de IA.
Integrantes	☑ **Sí**	Lista los nombres de pila del equipo de trabajo completo: Yoel, Ángel, Damián y Juan.
Roles	☑ **Sí**	Detalla las responsabilidades asignadas de forma equitativa para el frontend, backend, BD e IA.
Tecnologías	☑ **Sí**	Especifica el stack tecnológico base utilizado (Laravel 13, PHP 8.5, MariaDB, Node.js y Vite).
Requisitos	☑ **Sí**	Indica los requisitos mínimos necesarios en la máquina local para poder levantar el entorno.
Instalación	☑ **Sí**	Describe los comandos básicos de clonado de Git y la instalación de paquetes con Composer y npm.
Configuración	☑ **Sí**	Explica cómo duplicar el archivo .env.example para crear el entorno local sin revelar contraseñas.
Ejecución	☑ **Sí**	Muestra el comando php artisan serve para arrancar el servidor local en el puerto 8000.
Base de datos	☑ **Sí**	Aclara que las tablas se gestionan a través de las migraciones nativas de Laravel.
Estructura de documentación	☑ **Sí**	Mapear la ubicación de las carpetas principales del proyecto y de los archivos .md de la fase de diseño.
Estado actual del proyecto	☑ **Sí**	Especifica que el proyecto cierra la UT1 en fase de planificación, con el entorno validado y listo para codificar.

14. Procedimiento de reproducción del entorno

Este apartado debe permitir a otro miembro del grupo o al profesor reproducir el entorno en una máquina Ubuntu nueva. No incluir contraseñas reales.

Paso	Acción / comando	Resultado esperado
1	sudo apt update && sudo apt upgrade -y	Repositorios actualizados y paquetes del sistema operativo al día.
2	sudo snap install --classic code	Entorno de desarrollo Visual Studio Code instalado correctamente.
3	sudo apt install -y php-cli php-mbstring php-xml php-curl php-mysql	PHP configurado en el servidor junto a las extensiones necesarias para Laravel.
4	Descarga e instalación global de Composer moviendo el ejecutable a /usr/local/bin/composer	Gestor de dependencias de PHP operativo globalmente en la consola.
5	sudo apt install git -y	Herramienta de control de versiones Git lista para su uso.
6	sudo apt install nodejs npm -y	Entorno de ejecución Node.js y gestor de paquetes frontend operativos.
7	sudo apt install mariadb-server -y	Servidor de base de datos MariaDB/MySQL desplegado en la máquina.
8	sudo mysql -u root -e "CREATE DATABASE proyecto_laravel;"	Base de datos física del sistema creada y lista para el proyecto.
9	composer create-project laravel/laravel proyecto-laravel	Descarga e instalación limpia de la estructura base del framework Laravel.

14.1 Prueba final de reproducibilidad

Comprobación	Resultado
Otro integrante ha seguido el procedimiento	☑ **Sí** ☐ No
El proyecto Laravel se inicia correctamente	☑ **Sí** ☐ No
La conexión a BD puede configurarse	☑ **Sí** ☐ No
Se detectaron pasos que faltaban	**No faltaron pasos esenciales, pero tuvimos que corregir rutas en la terminal ya que por defecto se intentaba ejecutar Artisan fuera de la carpeta raíz del proyecto.**

15. Evidencias técnicas.

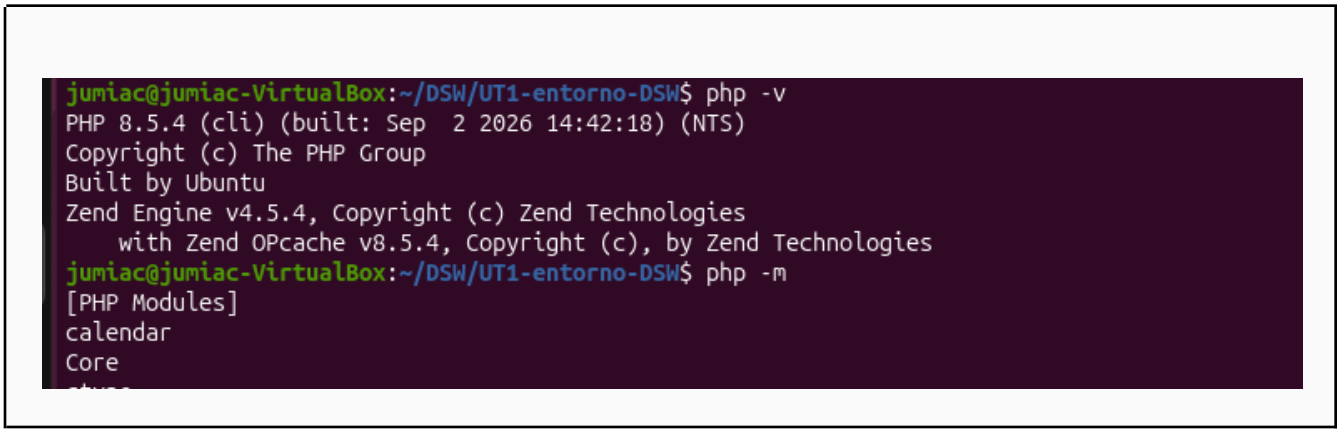
Evidencia 1. Ubuntu funcionando y versión del sistema

Campo	Contenido
Qué se demuestra	Que el sistema operativo base de la máquina virtual (Ubuntu) está correctamente instalado, operativo y actualizado en la versión exacta requerida para dar soporte al entorno.
Comando / acción realizada	lsb_release -a
Resultado observado	La terminal devuelve los datos del distribuidor mostrando de forma clara el identificador Ubuntu 26.04.1 LTS, lo que confirma que el sistema está listo y verificado.



Evidencia 2. PHP y extensiones necesarias

Campo	Contenido
Qué se demuestra	Que el intérprete de PHP está correctamente instalado en una versión compatible con Laravel y que los módulos críticos y extensiones (mbstring, xml, curl, pdo_mysql) están cargados y operativos en el servidor.
Comando / acción realizada	php -v (para la versión general) y php -m (para listar los módulos activos en la consola).
Resultado observado	La terminal devuelve que la versión del sistema es PHP 8.5.4 y muestra la lista completa de extensiones del núcleo cargadas correctamente, sin lanzar errores de dependencias faltantes.



Evidencia 3. Composer

Campo	Contenido
Qué se demuestra	Que el gestor de dependencias oficial de PHP (Composer) está correctamente instalado de forma global en el sistema operativo y responde de manera rápida y operativa para descargar paquetes de Laravel.
Comando / acción realizada	composer --version
Resultado observado	La consola responde correctamente devolviendo el texto con la versión exacta instalada en la máquina virtual, en este caso Composer versión 2.9.5.

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW$ composer --version
Composer version 2.9.5 2026-01-29 11:40:53
PHP version 8.5.4 (/usr/bin/php8.5)
Run the "diagnose" command to get more detailed diagnostics output.
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW$
```

Evidencia 4. Git

Campo	Contenido
Qué se demuestra	Que la herramienta de control de versiones Git está correctamente instalada en el sistema, lo que permite realizar el seguimiento de los cambios del código fuente y la sincronización con el repositorio remoto de GitHub.
Comando / acción realizada	git --version
Resultado observado	La terminal responde al instante mostrando la versión del software de control de versiones que se encuentra operativa en la máquina virtual, en este caso git versión 2.53.0.

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW$ git --version
git version 2.53.0
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW$
```

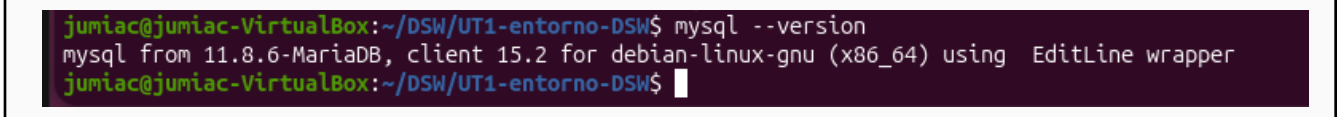
Evidencia 5. Node.js y npm

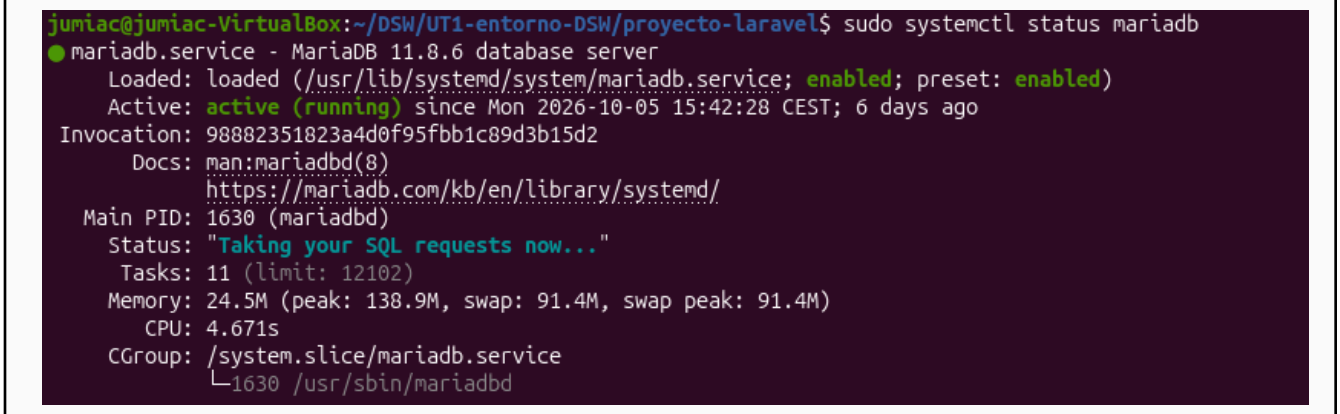
Campo	Contenido
Qué se demuestra	Que el entorno de ejecución Node.js y su gestor de paquetes (npm) están instalados correctamente, asegurando la capacidad de compilar los recursos de la interfaz visual (frontend) mediante Vite.
Comando / acción realizada	node --version && npm --version
Resultado observado	La consola responde de forma correcta mostrando las versiones activas del sistema en la máquina virtual: v24.12.0 para Node.js y 11.19.0 para npm.



Evidencia 6. Servidor de base de datos

Campo	Contenido
Qué se demuestra	Que el motor de bases de datos relacionales está instalado correctamente, permanece activo escuchando peticiones en el puerto estándar 3306 y permite accesos locales administrativos de forma totalmente operativa.
Comando / acción realizada	mysql --version (y la posterior comprobación interna de variables y base de datos con sudo mysql -u root).
Resultado observado	La terminal devuelve los datos del software del servidor indicando que la versión es 11.8.6-MariaDB, respondiendo a las consultas directas sin lanzar fallos de conexión.





```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ sudo mysql -u root -e "SHOW VARIABLES LIKE 'port'
; SELECT USER(); SHOW DATABASES;"
[sudo: authenticate] Contraseña:
+---------------+-------+
| Variable_name | Value |
+---------------+-------+
| port          | 3306  |
+---------------+-------+
+----------------+
| USER()         |
+----------------+
| root@localhost |
+----------------+
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$
```

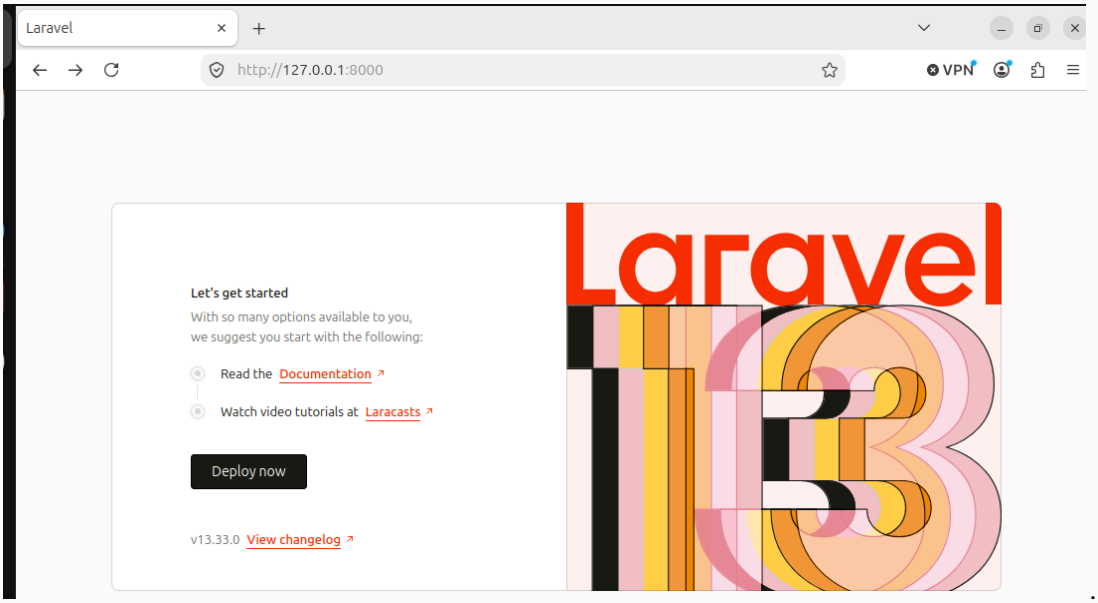
Evidencia 7. Laravel

Campo	Contenido
Qué se demuestra	Que el proyecto base de Laravel se ha creado de manera íntegra, posee la versión del framework esperada y su servidor local de desarrollo funciona perfectamente para servir la aplicación web.
Comando / acción realizada	php artisan --version (y el posterior arranque del entorno mediante php artisan serve).
Resultado observado	La consola devuelve de forma exitosa que se encuentra activo Laravel Framework 13.33.0 y levanta el servicio local sin problemas en el puerto habitual.

```
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$ php artisan --version
Laravel Framework 13.33.0
jumiac@jumiac-VirtualBox:~/DSW/UT1-entorno-DSW/proyecto-laravel$
```

Evidencia 8. Proyecto ejecutándose

Campo	Contenido
Qué se demuestra	Que el servidor local está respondiendo correctamente a las peticiones externas y que el navegador es capaz de interpretar y renderizar la vista de la primera página web de comprobación generada en Laravel.
Comando / acción realizada	Introducción de la URL de comprobación http://127.0.0 en la barra de direcciones del navegador web.
Resultado observado	Se visualiza correctamente la página web con el título del módulo, el mensaje de entorno preparado, los nombres de pila del equipo de trabajo y las versiones correctas de Laravel y PHP instaladas.



Evidencia 9. Página inicial de comprobación

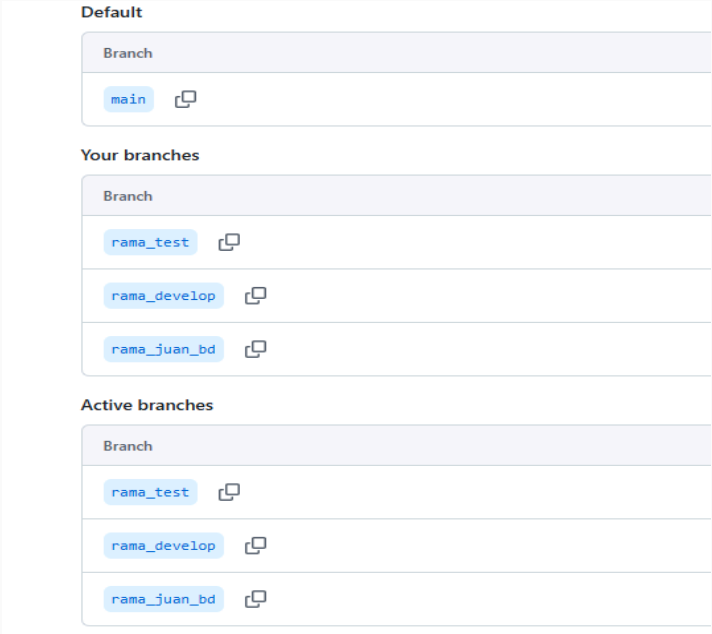
Campo	Contenido
Qué se demuestra	Que el motor de plantillas Blade de Laravel procesa correctamente las variables pasadas desde el servidor y renderiza la interfaz HTML específica solicitada para la validación de la asignatura.

Campo	Contenido
Comando / acción realizada	Carga y refresco de la vista dinámica en el navegador a través del direccionamiento de la ruta /dsw.
Resultado observado	Se muestra en pantalla la estructura del documento con los estilos correctos, mostrando el bloque con el mensaje "Entorno DSW funcionando correctamente" y la lista con el equipo de desarrollo.



Evidencia 10. Repositorio GitHub

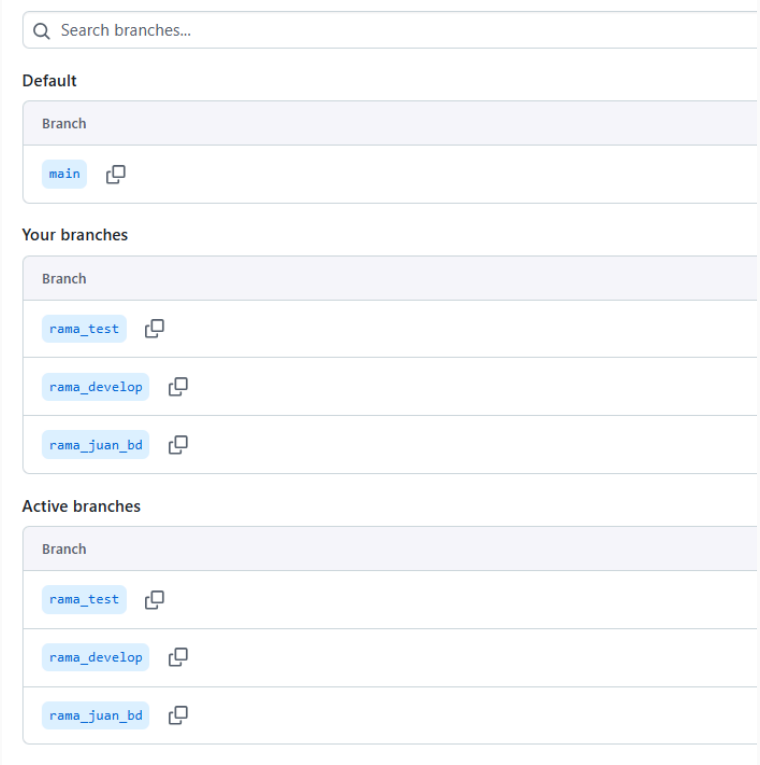
Campo	Contenido
Qué se demuestra	Que el repositorio remoto está correctamente creado en la plataforma GitHub, se encuentra configurado con visibilidad privada por seguridad y sincroniza todo el historial de commits y ramas del equipo.
Comando / acción realizada	Visualización directa de la página principal del repositorio en GitHub o ejecución del comando git status y git log en la consola local.
Resultado observado	Se muestra la interfaz web de GitHub con la estructura de archivos cargada, las ramas (main, rama_develop, rama_test, etc.) creadas y el archivo **README.md** oficial desplegado como portada principal.


Default
Branch
main
Your branches
Branch
rama_test
rama_develop
rama_juan_bd
Active branches
Branch
rama_test
rama_develop
rama_juan_bd


BD TABLAS Y OBJETOS MODIFICACIONES Y AMPLIACION last week
docs Subir documentación 1 3 hours ago
README.md Update README.md now
README
Plataforma de Simulación de Infraestructura Cloud con Asistente de IA
Esta plataforma web permite a los usuarios diseñar, presupuestar y simular el despliegue de arquitecturas Cloud de forma guiada, contando con el soporte de un asistente virtual basado en Inteligencia Artificial.
Este proyecto se desarrolla dentro del marco del módulo de Desarrollo de Aplicaciones Web (2º DAW / DSW).
Estado Actual (Final de la UT1)
Actualmente, el proyecto ha finalizado su Fase de Planificación, Arquitectura y Diseño Conceptual. El repositorio contiene los modelos de datos, diagramas de clases, la verificación del entorno correcto y la documentación técnica necesaria. La fase de codificación, creación de la base de datos e implementación comenzará de forma oficial en la UT2.
Stack Tecnológico Utilizado
Backend: Laravel 13.33.0 (PHP 8.5.4)
Base de Datos: MariaDB 11.8.6
Gestor de Dependencias: Composer 2.9.5 / npm 11.19.0
Compilador Frontend: Vite (Node.js v24.12.0)
Arquitectura del Servidor: Service Pattern (Desacoplamiento de lógica de negocio)
Integración de IA: API de Modelos de Lenguaje (LLM) con rol puramente consultivo
Control de Versiones: Git & GitHub
Estructura Modular del Sistema
La aplicación está diseñada de forma modular bajo una arquitectura limpia, dividida en 12 bloques funcionales:
1. Usuarios: Registro y control de accesos por roles (RBAC).
2. Catálogo: Servicios Cloud simulados y tarifas base.
3. Proveedores / Regiones: Zonas geográficas y latencias ficticias.
4. Proyectos: Entornos de trabajo de los clientes.
5. Requerimientos: Validación de necesidades previas al diseño.
6. Arquitecturas: Planos lógicos y topologías Cloud.
7. Carrito: Preselección temporal de recursos antes de la compra.
8. Presupuestos: Congelación de tarifas comerciales inmutables.
9. Pedidos: Formalización de la intención de compra basada en presupuestos aprobados.
10. Pagos: Simulación de transacciones financieras y rollback inmediato ante fallos.
11. Despliegues: Simulación asíncrona del aprovisionamiento técnico.

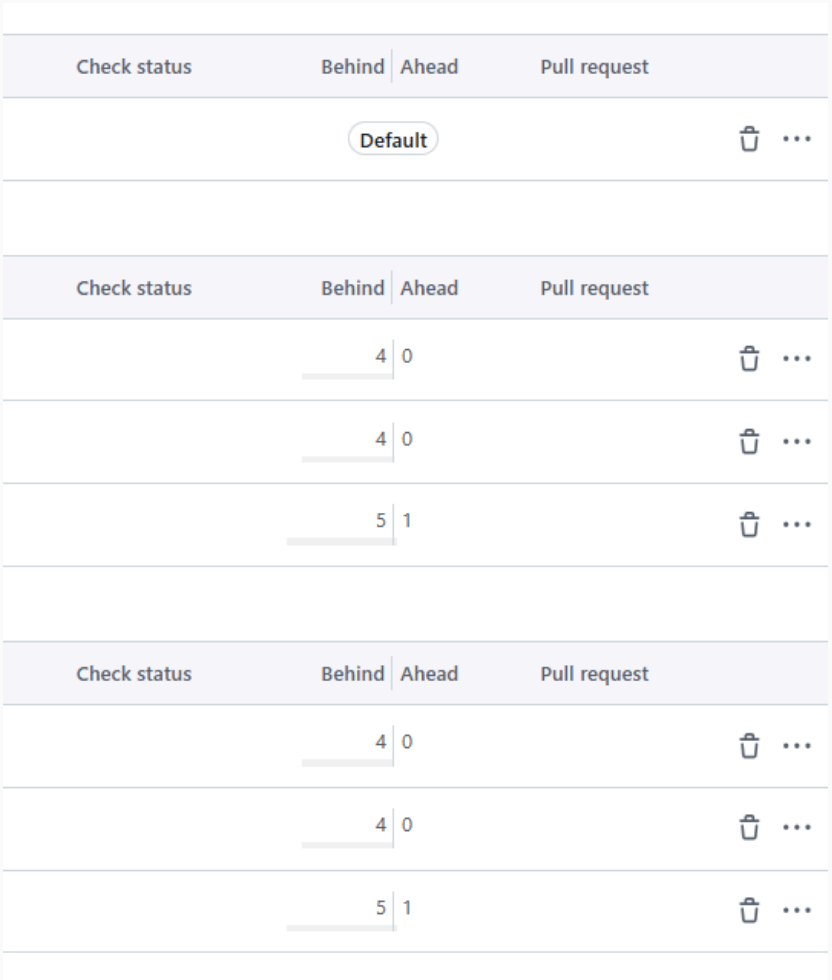
Evidencia 11. Ramas principales y feature

Campo	Contenido
Qué se demuestra	Que el equipo utiliza un flujo de trabajo estructurado en Git (GitFlow o similar) separando el código estable, las pruebas y los desarrollos de cada módulo mediante un árbol de ramas organizadas.
Comando / acción realizada	git branch -a en la terminal local o visualización de la pestaña "Branches" directamente en la interfaz web de GitHub.
Resultado observado	Se verifica la existencia y coexistencia de las ramas principales (main, rama_develop, rama_test) y las ramas de funcionalidad o personales de los integrantes sin conflictos de código.



Evidencia 12. Commits identificables de los integrantes

Campo	Contenido
Qué se demuestra	La participación real, equitativa y transparente de cada uno de los integrantes del grupo en el repositorio, demostrando el uso del estándar de mensajes Conventional Commits.
Comando / acción realizada	git log --oneline en la consola local o consulta de la pestaña "Commits" en el historial de la web de GitHub.
Resultado observado	Se visualiza el registro cronológico de aportaciones con autoría clara, mostrando prefijos como feat:, fix:, docs: y style: asociados a cada uno de los desarrolladores del equipo.



16. Incidencias encontradas y resolución

Incidencia	Síntoma / error	Diagnóstico	Solución	Verificación
Ejecución errónea del comando php artisan	Could not open input file: artisan en la terminal.	El comando se estaba lanzando desde la raíz del usuario (~), fuera de la carpeta del proyecto.	Cambiar de directorio con cd proyecto-laravel para situarse en la raíz correcta de la aplicación.	Al ejecutar php artisan --version devolvió de forma correcta el número del framework.
Error de sintaxis al comprobar los recursos de la máquina	La terminal muestra la ayuda técnica del comando free en lugar de los datos.	Se introdujo un espacio en blanco incorrecto entre el guion y el parámetro (free - h en lugar de free -h).	Corregir la sintaxis eliminando el espacio en blanco intermedio en la terminal de Ubuntu.	El comando free -h mostró de inmediato los Gigabytes reales de memoria RAM.
Bloqueo del gestor de paquetes apt al actualizar	Error de *lock* o recurso ocupado al hacer sudo apt update.	El sistema operativo Ubuntu estaba lanzando actualizaciones automáticas en segundo plano al arrancar la máquina virtual.	Esperar unos minutos a que finalizara la tarea automática del sistema o reiniciar el proceso de red secundario.	El comando sudo apt update volvió a ejecutarse de manera limpia y sin res

17. Conclusiones técnicas

- **¿Qué parte del entorno fue más difícil de reproducir y por qué?**
  La parte más compleja fue asegurar la correcta configuración y el orden de las dependencias globales en la máquina virtual, asegurando que los comandos de Artisan se ejecutaran exactamente desde el directorio raíz del proyecto y no desde la carpeta de usuario.
- **¿Qué dependencia existe entre PHP, Composer y Laravel?**
  La dependencia es total y en cadena: PHP es el lenguaje de programación base que ejecuta todo el código; Composer es el gestor que descarga y organiza los paquetes; y Laravel es el framework que se construye utilizando esas dependencias de PHP para funcionar.
- **¿Qué papel tienen Node.js/npm en este proyecto?**
  Aunque el núcleo del proyecto funciona en el backend con PHP, Node.js y npm son indispensables en el entorno de desarrollo para gestionar, compilar y optimizar todos los recursos y librerías de la interfaz visual (frontend) mediante Vite.
- **¿Qué medidas han aplicado para evitar publicar secretos?**
  Se ha configurado de forma estricta el archivo .gitignore para bloquear la subida del fichero .env al repositorio de GitHub, manteniendo las contraseñas e identificadores reales a nivel local y compartiendo únicamente la plantilla vacía .env.example.

- **¿Qué aporta Git/GitHub al trabajo colaborativo del equipo?**
  Aporta un control total del historial del código, permitiendo que los cuatro integrantes del equipo (**Yoel, Ángel, Damián y Juan**) trabajen en paralelo en diferentes ramas sin pisarse los cambios, facilitando la revisión visual del código antes de integrarlo.
- **¿Está el entorno preparado para comenzar la implementación en la siguiente UT? Justificar.**
  Sí, está completamente preparado. Se ha verificado por terminal que todos los servicios críticos (PHP, MariaDB, Node.js y Composer) están instalados en sus versiones correctas, el repositorio compartido está operativo y la ruta inicial de Laravel responde perfectamente en el navegador local.

18. Checklist antes de entregar

Comprobaciones técnicas	Comprobaciones de entrega
☑ Ubuntu está identificado y actualizado.	☑ El repositorio contiene main, develop y test, además de ramas feature cuando proceda.
☑ VS Code está instalado y las extensiones están justificadas.	☑ Todos los integrantes tienen aportaciones identificables.
☑ PHP funciona y sus extensiones están documentadas.	☑ .env no está publicado.
☑ Composer funciona.	☑ .gitignore se ha revisado.
☑ Git funciona.	☑ .env.example no contiene secretos.
☑ Node.js y npm funcionan.	☑ La carpeta docs existe.
☑ El servidor de BD está operativo.	☑ README.md incluye todos los apartados obligatorios.
☑ Laravel funciona y php artisan --version responde.	☑ Las capturas son legibles y están explicadas.
☑ La página inicial de comprobación es accesible.	☑ El procedimiento de reproducción puede ser seguido por otra persona.